

МИНОБРНАУКИ РОССИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

**«САНКТ-ПЕТЕРБУРГСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ ПЕТРА
ВЕЛИКОГО»**

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление 02.03.01 Математика и компьютерные науки

Отчёт по дисциплине «Математическая логика и теория автоматов»

Лабораторная работа №3 «Построение
контекстно-свободной грамматики
подмножества естественного языка»

Вариант: Präsens - простое настоящее время в немецком языке

Студент: _____

Черданцев Артём Сергеевич

Преподаватель: _____

Востров Алексей Владимирович

«____» _____ 20__ г.

Санкт-Петербург, 2025

Содержание

Введение	3
1 Математическое описание	4
1.1 Описание выбранной грамматики	4
1.2 Правила построения выбранной грамматики	4
1.3 Математическое описание порождающей грамматики	4
1.4 Пример дерева для грамматики	5
2 Особенности реализации	7
2.1 Класс Validator	7
2.2 Класс Generator	9
2.3 Класс Node	10
3 Результаты работы программы	12
Заключение	14
Список источников	15

Введение

В данной лабораторной работе необходимо:

1. Выбрать подмножество естественного языка, в качестве грамматики для построения;
2. Согласовать выбранную грамматику с преподавателем;
3. Реализовать распознавание вводимого предложения на принадлежность грамматике;
4. Реализовать генератор предложений в соответствии с выбранной грамматикой.

1 Математическое описание

1.1 Описание выбранной грамматики

Для данной работы была выбрана грамматика: **Präsens** (настоящее время) немецкого языка.

Используется **Präsens** в немецком языке для

- выражения действий, которые происходят в настоящем времени (что делаю?).
- донесения информации о действиях, которые произойдут в ближайшем будущем.
- передачи систематических действий, фактов и явлений.

В данной работе реализованы действия с утвердительными, отрицательными и вопросительными предложениями в **Präsens**.

1.2 Правила построения выбранной грамматики

Утвердительные предложения строятся по следующему правилу:

Subject + Verb + Придаточная часть (опционально) + "."

Отрицательные предложения строятся по следующему правилу:

Subject + Verb + nicht + Придаточная часть (опционально) + "."

Вопросительные предложения строятся по следующему правилу:

Verb/Special question + Subject + Придаточная часть (опционально) + "?"

1.3 Математическое описание порождающей грамматики

Порождающей грамматикой Хомского G называется множество: $\{T, N, S, R\}$, где:

- T - конечное множество символов (терминальный словарь);
- N - конечное множество символов (нетерминальный словарь);
- $S \in N$ - начальный нетерминал;
- R - конечное множество правил (продукций) вида $\alpha \rightarrow \beta$, где α и β цепочки над словарём $T \cup N$

Контекстно-свободной грамматикой называется грамматика, у которой левые части всех продукций являются одиночными нетерминалами. То есть все правила грамматики имеют вид:

$$A \rightarrow \lambda, \text{ где } A \in T, \lambda \in N$$

Для построения грамматики **Präsens** было выбрано:

$T = \{\text{schön, groß, klein, neu, alt, klug, freundlich, stark, schwach, rot, blau, grün, schwarz, weiß, lang, kurz, gut, schlecht, wichtig, interessant, freundlich, fantastisch, apfel, orange, banane, buch, tisch, stift, arzt, lehrer, ingenieur, künstler, notizbuch, papier, computer, mensch, hund, katze, giraffe, löwe, welt, kind, frau, platz, arbeit, woche, regierung, firma, er, sie, es, wir, sie, ich, glaubt, hält, fühlt, respektiert, versteht, hört, findet, stellt, bewundert, kommt, läuft, fährt, reist, steht, sieht, spricht, rennt, tanzt, singt, lacht, weint, von, zu, in, Berlin, München, Hamburg, Frankfurt, Köln, Düsseldorf, Stuttgart, Leipzig, Dresden, selbstbewusst, schlecht, gut, schnell, langsam, schön, und, aber}\}$;

$N = \{ \text{Sentence, SentenceP, SentenceN, SentenceQ, SubjectGroup, vGroup, ComplexPart, PlacesPart, AdjAction, Adjective, NounObject, Pronoun, vActGroup, vStateGroup, vMoveGroup, Link, Preposition, Place} \};$

$S = \text{Sentence};$

$R = \{$

$\langle \text{Sentence} \rangle ::= \langle \text{SentenceP} \rangle \mid \langle \text{SentenceN} \rangle \mid \langle \text{SentenceQ} \rangle$

$\langle \text{SentenceP} \rangle ::= \langle \text{SubjectGroup} \rangle \langle \text{vGroup} \rangle \langle \text{AdjAction} \rangle \langle \text{ComplexPart} \rangle \langle \text{PlacesPart} \rangle.$

$\langle \text{SentenceN} \rangle ::= \langle \text{SubjectGroup} \rangle \langle \text{vGroup} \rangle \langle \text{AdjAction} \rangle \text{ nicht } \langle \text{PlacesPart} \rangle.$

$\langle \text{SentenceQ} \rangle ::= \langle \text{vGroup} \rangle \langle \text{AdjAction} \rangle \langle \text{SubjectGroup} \rangle \langle \text{PlacesPart} \rangle?$

$\langle \text{SubjectGroup} \rangle ::= \langle \text{Adjective} \rangle \langle \text{NounObject} \rangle \mid \langle \text{Pronoun} \rangle$

$\langle \text{vGroup} \rangle ::= \langle \text{vActGroup} \rangle \mid \langle \text{vStateGroup} \rangle \mid \langle \text{vMoveGroup} \rangle$

$\langle \text{ComplexPart} \rangle ::= \langle \text{Link} \rangle \langle \text{SubjectGroup} \rangle \langle \text{vGroup} \rangle \langle \text{AdjAction} \rangle \mid \varepsilon$

$\langle \text{PlacesPart} \rangle ::= \langle \text{Preposition} \rangle \langle \text{Place} \rangle \mid \varepsilon$

$\langle \text{AdjAction} \rangle ::= \text{selbstbewusst} \mid \text{schlecht} \mid \text{gut} \mid \text{schnell} \mid \text{langsam} \mid \text{schön} \mid \varepsilon$

$\langle \text{Adjective} \rangle ::= \text{schön} \mid \text{groß} \mid \text{klein} \mid \text{neu} \mid \text{alt} \mid \text{klug} \mid \text{freundlich} \mid \text{stark} \mid \text{schwach} \mid \text{rot} \mid \text{blau} \mid \text{grün} \mid \text{schwarz} \mid \text{weiß} \mid \text{lang} \mid \text{kurz} \mid \text{gut} \mid \text{schlecht} \mid \text{wichtig} \mid \text{interessant} \mid \text{freundlich} \mid \text{fantastisch}$

$\langle \text{NounObject} \rangle ::= \text{apfel} \mid \text{orange} \mid \text{banane} \mid \text{buch} \mid \text{tisch} \mid \text{stift} \mid \text{arzt} \mid \text{lehrer} \mid \text{ingenieur} \mid \text{künstler} \mid \text{notizbuch} \mid \text{papier} \mid \text{computer} \mid \text{mensch} \mid \text{hund} \mid \text{katze} \mid \text{giraffe} \mid \text{löwe} \mid \text{welt} \mid \text{kind} \mid \text{frau} \mid \text{platz} \mid \text{arbeit} \mid \text{woche} \mid \text{regierung} \mid \text{firma}$

$\langle \text{Pronoun} \rangle ::= \text{er} \mid \text{sie} \mid \text{es} \mid \text{wir} \mid \text{sie} \mid \text{ich}$

$\langle \text{vActGroup} \rangle ::= \text{steht} \mid \text{sieht} \mid \text{spricht} \mid \text{rennt} \mid \text{tanzt} \mid \text{singt} \mid \text{lacht} \mid \text{weint}$

$\langle \text{vStateGroup} \rangle ::= \text{glaubt} \mid \text{hält} \mid \text{fühlt} \mid \text{respektiert} \mid \text{versteht} \mid \text{hört} \mid \text{findet} \mid \text{stellt} \mid \text{bewundert}$

$\langle \text{vMoveGroup} \rangle ::= \text{kommt} \mid \text{läuft} \mid \text{fährt} \mid \text{reist}$

$\langle \text{Link} \rangle ::= \text{und} \mid \text{aber}$

$\langle \text{Preposition} \rangle ::= \text{von} \mid \text{zu} \mid \text{in}$

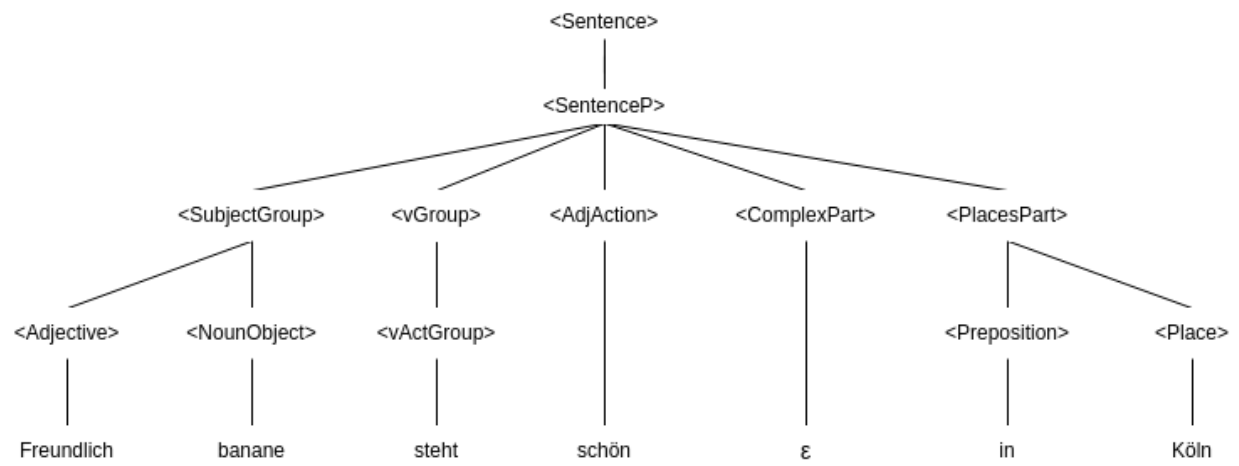
$\langle \text{Place} \rangle ::= \text{Berlin} \mid \text{München} \mid \text{Hamburg} \mid \text{Frankfurt} \mid \text{Köln} \mid \text{Düsseldorf} \mid \text{Stuttgart} \mid \text{Leipzig} \mid \text{Dresden}$

$\}.$

1.4 Пример дерева для грамматики

Для иллюстрации работы представленной грамматики рассмотрим синтаксическое дерево для предложения:

Freundlich banana steht schön in Köln



2 Особенности реализации

Для реализации конечного автомата была написана программа на Java.

2.1 Класс Validator

Класс Validator - класс программы, который проверяет валидность введенной пользователем строки для выбранной грамматики.

1. **Функция NextWord** отвечает за переход к следующему слову во входной строке.

Вход: Внутренний список токенов и текущий индекс разбора.

Выход: Обновлённое значение текущего слова, либо исключение при выходе за пределы списка.

2. **Функция SentenceP** реализует синтаксический анализ утвердительного предложения.

Вход: Список токенов утвердительного предложения.

Выход: Исключение при ошибке разбора, либо успешное завершение при корректной структуре.

3. **Функция SentenceQ** реализует синтаксический анализ вопросительного предложения.

Вход: Список токенов вопросительного предложения.

Выход: Исключение при ошибке разбора, либо успешное завершение при корректной структуре.

4. **Функция SentenceN** реализует синтаксический анализ отрицательного предложения.

Вход: Список токенов отрицательного предложения.

Выход: Исключение при ошибке разбора, либо успешное завершение при корректной структуре.

```
1 public class Validator {
2     Dictionary dict;
3     int curr_index;
4     ArrayList<String> tokens;
5     String currWord;
6
7     public Validator() {
8         dict = new Dictionary();
9     }
10
11     void NextWord() throws RuntimeException {
12         curr_index++;
13         if (curr_index < tokens.size())
14             currWord = tokens.get(curr_index);
15         else throw new RuntimeException("9");
16     };
17
18     void SentenceP() throws RuntimeException {
```

```

19     curr_index=0;
20     currWord=tokens.get(0);
21     SubjectGroup();
22     vGroup();
23     AdjAction();
24     if (currWord.equals(".")) return;
25
26     else {
27         Link();
28         SubjectGroup();
29         vGroup();
30         AdjAction();
31     }
32     if (currWord.equals(".")) return;
33
34     else {
35         Preposition();
36         Place();
37     }
38     if (currWord.equals(".")) return; else throw new RuntimeException("0");
39 }
40
41 void SentenceQ() throws RuntimeException {
42     curr_index=0;
43     currWord=tokens.get(0);
44     vGroup();
45     SubjectGroup();
46     if (currWord.equals("?")) return;
47
48     else {
49         Preposition();
50         Place();
51     }
52     if (currWord.equals("?")) return; else throw new RuntimeException("0");
53 }
54
55 void SentenceN() throws RuntimeException {
56     curr_index=0;
57     currWord=tokens.get(0);
58     SubjectGroup();
59     vGroup();
60     NextWord();
61     if (currWord.equals(".")) return;
62
63     else {
64         Preposition();
65         Place();
66     }
67     if (currWord.equals(".")) return; else throw new RuntimeException("0");
68 }

```


69 ...
70 }

2.2 Класс Generator

Класс Generator - класс программы, который генерирует предложения по правилам описанным в разделе 1.3.

1. Создается структура helpVec, которая определяет различные шаблоны для утвердительных предложений. Каждый элемент Vector<Node> представляет определенную последовательность грамматических компонентов (например, SubjectGroup, vGroup, AdjAction).
2. Метод SentP.setMatrix_roles(...) задаёт структуру шаблонов для генерации утвердительных предложений.

Вход: Вектор шаблонов предложений (в виде Vector<Vector<Node>»).

Выход: Внутренняя инициализация шаблонов в генераторе утверждений.

```
1 public class Generator {  
2  
3     Generator() {  
4         Dictionary dict = new Dictionary();  
5  
6         // Определение структуры утвердительного предложения  
7         Vector<Vector<Node>> helpVec = new Vector<Vector<Node>>() {{  
8             add(new Vector<Node>() {{  
9                 add(SubjectGroup);  
10                add(vGroup);  
11                add(AdjAction); //простое  
12            }});  
13            add(new Vector<Node>() {{  
14                add(SubjectGroup);  
15                add(vGroup);  
16                add(AdjAction);  
17  
18                add(Link);  
19                add(SubjectGroup);  
20                add(vGroup);  
21                add(AdjAction); //сложное  
22            }});  
23            add(new Vector<Node>() {{  
24                add(SubjectGroup);  
25                add(vGroup);  
26                add(AdjAction);  
27                add(Link);  
28                add(SubjectGroup);  
29                add(vGroup);  
30                add(AdjAction);  
31                add(Preposition); //сложное с местом  
32                add(Place);
```

```

33         });
34         add(new Vector<Node>() {{
35             add(SubjectGroup);
36             add(vGroup);
37             add(AdjAction);
38             add(Preposition); //простое с местом
39             add(Place);
40         }});
41     }};
42     SentP.setMatrix_roles(new Vector<>(helpVec));
43     helpVec.clear();
44     ...
45 }

```

2.3 Класс Node

Класс Node - подкласс Generator, использующийся для реализации нетерминалов разделённых "(см. раздел 1.3.) и позволяет выполнять случайные выборки элементов для терминалов.

- void setTokens(ArrayList<String> tokens): устанавливает возможные токены для данного узла. void setMatrix_roles(Vector<Vector<Node>> matrix_roles): задает возможные комбинации дочерних узлов.
- String getRandomToken(): выбирает случайный терминал из списка токенов. Vector<Node> getRandomRules(): выбирает случайную последовательность узлов для построения предложения.
- Конструктор Node(String name): инициализирует узел с именем и пустыми структурами для ролей и токенов.

```

1  class Node {
2      String name;
3      Vector<Vector<Node>> matrix_roles;
4      ArrayList<String> tokens;
5
6      public void setTokens(ArrayList<String> tokens) {
7          this.tokens = tokens;
8      }
9
10     public void setMatrix_roles(Vector<Vector<Node>> matrix_roles) {
11         this.matrix_roles = matrix_roles;
12     }
13
14     String getRandomToken() {
15         return tokens.get(new Random().nextInt(tokens.size()));
16     }
17
18     Vector<Node> getRandomRules() {
19         return matrix_roles.get(new Random().nextInt(matrix_roles.size()));
20     }
21 }

```

```
22     Node(String name) {
23         this.name = name;
24         matrix_roles = new Vector<>>();
25         tokens = new ArrayList<>>();
26     }
27 }
```

3 Результаты работы программы

Результат генерации случайных утвердительного, отрицательного и вопросительного предложений представлен на рис. 1

```

      MENU
*****
[M] - Напечатать MENU
[1] - Ввести предложение для проверки
[2] - Сгенерировать предложение
[0] - Завершить работу программы
*****

Выберите вариант: 2
////////////////////
(1) - Генерация утвердительного предложения
(2) - Генерация отрицательного предложения
(3) - Генерации вопросительного предложения
Выберите какое предложение сгенерировать: 1
Сгенерированная цепочка: Lang tisch versteht schlecht aber wir stellt langsam.
////////////////////

Выберите вариант: 2
////////////////////
(1) - Генерация утвердительного предложения
(2) - Генерация отрицательного предложения
(3) - Генерации вопросительного предложения
Выберите какое предложение сгенерировать: 2
Сгенерированная цепочка: Stark löwe kommt nicht von Hamburg.
////////////////////

Выберите вариант: 2
////////////////////
(1) - Генерация утвердительного предложения
(2) - Генерация отрицательного предложения
(3) - Генерации вопросительного предложения
Выберите какое предложение сгенерировать: 3
Сгенерированная цепочка: Respektiert ich?
////////////////////

Выберите вариант: █
```

Рис. 1: Результат генерации случайных предложений

Результат проверки введенного пользователем предложения представлен на рис. 2

```

Выберите вариант: 1
////////////////////
Введите предложение для проверки: Wir kommt langsam nicht.
Предложение принадлежит грамматике! Это отрицательное предложение в настоящем времени!
////////////////////
```

Рис. 2: Результат проверки введенного валидного предложения

Пример вывода сообщений об ошибках во введенных предложениях представлен на рис.

```
Выберите вариант: 1
////////////////////////
Введите предложение для проверки:
Пустая строка не принадлежит грамматике!
////////////////////////
```

Рис. 3: Вывод ошибки пустого предложения

```
Выберите вариант: 1
////////////////////////
Введите предложение для проверки: Wir.
Глаголы содержат ошибку.
////////////////////////
```

Рис. 4: Вывод ошибки знаков препинания

```
Выберите вариант: 1
////////////////////////
Введите предложение для проверки: Wir kommt nicht
Предложение должно заканчиваться точкой или вопросительным знаком! Попробуйте снова.
////////////////////////
```

Рис. 5: Вывод ошибки синтаксиса

Заключение

В ходе выполнения работы была разработана контекстно-свободная грамматика, охватывающая подмножество предложений в Präsens (настоящее время) немецкого языка. Грамматика описывает синтаксически корректные конструкции, включающие утвердительные, отрицательные и вопросительные предложения. При этом, в целях упрощения, семантическая согласованность (времена, лица, смысловые связи) не учитывалась — грамматика ориентирована исключительно на формальный синтаксический анализ.

На основе данной грамматики была реализована программная система, включающая следующие основные функции:

1. Генерация предложений Система способна автоматически формировать случайные синтаксически правильные предложения, опираясь на заданные шаблоны структуры. Генерация основана на шаблонах узлов (Node), представляющих основные компоненты предложения — подлежащее, сказуемое, дополнение, обстоятельство и т.д.
2. Синтаксическая проверка предложений Программа предоставляет механизм валидации введенных пользователем предложений. При проверке используется пошаговый разбор структуры: выявление подлежащего, глагольной группы, обстоятельства, предлога и т.п. Для каждого типа предложения реализован свой метод разбора: SentenceP (утвердительное), SentenceN (отрицательное), SentenceQ (вопросительное).
3. Обработка различных типов предложений Разработка охватывает три типа синтаксических конструкций:
 - (a) утвердительные предложения (простые и составные),
 - (b) отрицательные предложения,
 - (c) вопросительные предложения в инверсивной форме.

Преимущества:

- Использование отдельного файла для хранения нетерминалов и соответствующих им терминалов, что упрощает дальнейшее изменение этого файла.
- Применение метода рекурсивного спуска для валидации предложения на соответствие грамматическим правилам.
- При вводе предложений на проверку, если в какой-то части предложения обнаружены ошибки, программа выведет соответствующее сообщение.

Недостатки:

- Использование разных контейнеров: ArrayList и Vector для одних и тех же целей (хранение строк). Это делает программу более сложной к восприятию и усложняет масштабирование.

Масштабирование:

- Программу можно расширить, добавив новые терминальные и нетерминальные символы. Для этого потребуется обновить содержимое контейнеров в классе Dictionary и добавить соответствующие методы в классах Validator и Generator.
- Можно добавить генерацию и обработку предложений с более сложной структурой

Список источников

1. Ф.А Новиков. Дискретная математика для программистов: Учебник для вузов. 3-е изд. — СПб: Питер, 2008г. 384 с.
2. А.В. Востров. Лекция №8 по дисциплине «Математическая логика и теория автоматов». 2024г. 52 сл.